



Unión Europea

Fondo Social Europeo

El FSE invierte en tu futuro

MVC

Modelo, Vista, Controlador

Pepe

INSTITUT



**GENERALITAT
VALENCIANA**

Conselleria d'Educació,
Investigació, Cultura i Esport



GOBIERNO
DE ESPAÑA

MINISTERIO
DE EDUCACIÓN, CULTURA
Y DEPORTE

Continguts

1	Modelo Vista Controlador	3
2	Conexión	4
3	El Modelo (Models)	5
4	El Controlador	7
5	La Vista	8
6	Login & Password	9
7	Ejercicios	11
7.1	Ejercicio 1	11

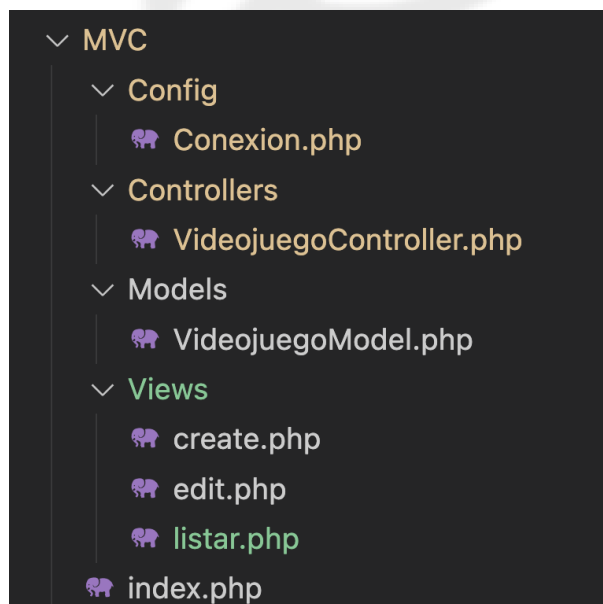
1 Modelo Vista Controlador

Existen 3 niveles de abstracción:

1. Modelo.- Es quien define la lógica de negocio. Son las clases y los métodos que se comunican directamente con la base de datos.
2. Vista.- Muestra la información al usuario de manera lógica y legible.
3. Controlador.- Es el intermediario entre la vista y el modelo. Controla las interacciones del usuario en la vista. Pide los datos al modelo y los devuelve a la vista para que los muestre. Es el encargado de realizar las llamadas a las clases y los métodos.

- Funcionamiento del MVC

- El usuario realiza una petición.
- El controlador captura la petición.
- El controlador hace la llamada al modelo correspondiente.
- El modelo interactúa con la base de datos.
- El controlador recibe la información del modelo (base de datos) y la envía a la vista.
- La vista muestra la información. Siguiendo con el ejemplo del tema anterior, *videojuego*, vamos a ver la estructura con la imagen siguiente:



(a) Estructura

- **Carpeta Models** donde tendremos la información de cada tabla de nuestra BBDD, en el ejemplo tendremos la tabla *videojuegos*.

- **Carpeta Controller** donde tendremos los métodos para actualizar los datos del modelo y redireccionar a la vista.
- **Carpeta Views** Donde residen las interfaz con el usuario.
- **Carpeta Config** Conexión con la BBDD.
- Vemos un fichero *index.php* fuera de la estructura que será el primero que se llamará y lo que hace es cargar el *controller*.

2 Conexión

Primero vamos a crear un archivo llamado `Conexion.php` en este archivo vamos a conectarnos con el sistema gestor de base de datos que en este caso es mysql, para podernos conectar a mysql necesitamos sus características, en este caso:

- Driver: que es el nombre del sistema gestor de base de datos.
- host: en este caso el host es localhost.
- usuario: Mi caso uso el root. Sino tengo una contraseña dejamos vacío el campo.
- dbName: El nombre de nuestra base de datos.

Ahora vamos a crear una instancia de `PDO`.

```
// MYSQL: Conexión con la base de datos
class Conexion {
    private $pdo;
    private $host;
    private $nombreBD ;
    private $usuario;
    private $password ;

    public function __construct()
    {
        $this->pdo="";
        $this->host = "localhost";
        $this->nombreBD = "videojuego";
        $this->usuario = "root";
        $this->password = "";
    }

    public function conectar(){

        try{

            $this->pdo = new PDO("mysql:host=$this->host;dbname=$this->
                                nombreBD;charset=utf8",$this->usuario, $this->password);
```

```
        return $this->pdo;
    } catch (PDOException $e) {
        print " <p class=\"aviso\">Error: No puede conectarse con
            la base de datos. {$e->getMessage()}</p>\n";
        exit;
    }
}
}
```

3 El Modelo (Models)

El modelo es quien define la lógica de negocio. Son las clases y los métodos que se comunican directamente con la base de datos. Creamos nuestra clase en mi caso `Videojuego.php`, donde tendremos que acceder vamos a `Conexion.php`.

```
<?php

require_once "Config/Conexion.php";

class Videojuego
{
    private $pdo;
    public function __construct() {
        $database = new Conexion();
        $this->pdo = $database->conectar();
    }
    // a partir de aquí iran todas los métodos CRUD

    public function getAll()
    {
        try{
            $query = "SELECT * FROM videojuegos";
            $registro = $this->pdo->prepare($query);
            $registro->execute();
            return $registro->fetchAll();
        }catch (PDOException $e)
        {
            die($e->getMessage());
        }
    }

    public function getId($id)
    {
        try{
            $query = "SELECT * FROM videojuegos WHERE id = $id";
            $registro = $this->pdo->prepare($query);
            $registro->execute();
            return $registro->fetch();
        }catch (PDOException $e)
```

```
{
    die($e->getMessage());
}
}
public function delete($d)
{
    try{
        $insercion = $this->pdo->prepare("delete from videojuegos where
            id=:id");
        $insercion->bindParam(':id', $d);
        return $insercion->execute();
    }catch(PDOException $e)
    {
        die($e->getMessage());
    }
}
public function edit($i,$t,$g,$p)
{
    try{
        $insercion = $this->pdo->prepare("update videojuegos set titulo=:
            titulo, genero=:genero, precio=:precio where id=:id");
        $insercion->bindParam(':id', $i);
        $insercion->bindParam(':titulo', $t);
        $insercion->bindParam(':genero', $g);
        $insercion->bindParam(':precio', $p);
        $insercion->execute();
        return true;
    }catch (PDOException $e)
    {
        die($e->getMessage());
    }
}
public function save($t,$g,$p)
{
    try{
        $insercion = $this->pdo->prepare("INSERT INTO videojuegos(titulo,
            genero,precio) VALUES(:titulo, :genero, :precio)");
        $insercion->bindParam(':titulo', $t);
        $insercion->bindParam(':genero', $g);
        $insercion->bindParam(':precio', $p);
        return $insercion->execute();
    }catch (PDOException $e)
    {
        die($e->getMessage());
    }
}
public function update($id, $titulo, $genero, $precio) {
    $query = "UPDATE videojuegos SET titulo = :titulo, genero = :genero,
        precio = :precio WHERE id = :id";
    $insercion = $this->pdo->prepare($query);
    $insercion->bindParam(":id", $id);
```

```
$insercion->bindParam(":titulo", $titulo);  
$insercion->bindParam(":genero", $genero);  
$insercion->bindParam(":precio", $precio);  
return $insercion->execute();  
}  
}
```

Como podéis observar las funciones de los métodos son las mismas que las estudiadas en el tema anterior. Para que la clase se ajuste más al paradigma de la POO debería tener los *getter* y *setter* de los campos de la tabla y crear dichos campos como variables privadas. También sería conveniente a la clase podenerle el *sufijo* del modelo, por ejemplo *VideojuegoController* o *VideojuegoModel*

4 El Controlador

Es el intermediario entre la vista y el modelo. Controla las interacciones del usuario en la vista. Pide los datos al modelo y los devuelve a la vista para que los muestre. Es el encargado de realizar las llamadas a las clases y los métodos.

```
<?php  
require_once "Models/VideojuegoModel.php";  
  
class VideojuegoController {  
  
    private $videojuegoModel;  
  
    public function __construct() {  
        $this->videojuegoModel = new Videojuego();  
    }  
  
    public function index() {  
        $videojuegos = $this->videojuegoModel->getAll();  
        require "Views/listar.php";  
    }  
  
    public function create() {  
        if ($_SERVER['REQUEST_METHOD'] == 'POST') {  
            $titulo = $_POST['titulo'];  
            $genero = $_POST['genero'];  
            $precio = $_POST['precio'];  
            $this->videojuegoModel->save($titulo, $genero, $precio);  
            header("Location: index.php");  
        } else {  
            require "Views/create.php";  
        }  
    }  
  
    public function edit($id) {
```

```
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $titulo = $_POST['titulo'];
    $genero = $_POST['genero'];
    $precio = $_POST['precio'];
    $this->videojuegoModel->update($id, $titulo, $genero, $precio);
    ;
    header("Location: index.php");
} else {
    $videojuego = $this->videojuegoModel->getById($id);
    require "Views/edit.php";
}

}

public function delete($id) {
    $this->videojuegoModel->delete($id);
    header("Location: index.php");
}

}
```

5 La Vista

Muestra la información al usuario de manera lógica y legible.

listar.php

```
<h1>Lista de Videojuegos</h1>
<a href="index.php?action=create">Agregar Videojuego</a>
<table border="1">
    <tr>
        <th>ID</th>
        <th>Título</th>
        <th>Género</th>
        <th>Precio</th>
        <th>Acciones</th>
    </tr>
    <?php foreach ($videojuegos as $videojuego): ?>
        <tr>
            <td><?= $videojuego['id'] ?></td>
            <td><?= $videojuego['titulo'] ?></td>
            <td><?= $videojuego['genero'] ?></td>
            <td><?= $videojuego['precio'] ?></td>
            <td>
                <a href="index.php?action=edit&id=<?= $videojuego['id'] ?>">
                    Editar</a>
                <a href="index.php?action=delete&id=<?= $videojuego['id'] ?>"
                    onclick="return confirm('¿Estás seguro?')">Eliminar
                </a>
            </td>
        </tr>
    </?php>
</table>
```

```
        </td>
    </tr>
<?php endforeach; ?>
</table>
```

edit.php

```
<h1>Editar Videojuego</h1>
<form method="POST" action="index.php?action=edit&id=<?= $videojuego['id']
?>">
    <label for="titulo">Título:</label>
    <input type="text" name="titulo" value="<?= $videojuego['titulo'] ?>"
        required>
    <br>
    <label for="genero">Género:</label>
    <input type="text" name="genero" value="<?= $videojuego['genero'] ?>"
        required>
    <br>
    <label for="precio">Precio:</label>
    <input type="number" name="precio" value="<?= $videojuego['precio'] ?>"
        " step="0.01" required>
    <br>
    <button type="submit">Actualizar</button>
</form>
```

create.php

```
<h1>Agregar Videojuego</h1>
<form method="POST" action="index.php?action=create">
    <label for="titulo">Título:</label>
    <input type="text" name="titulo" required>
    <br>
    <label for="genero">Género:</label>
    <input type="text" name="genero" required>
    <br>
    <label for="precio">Precio:</label>
    <input type="number" name="precio" step="0.01" required>
    <br>
    <button type="submit">Guardar</button>
</form>
```

6 Login & Password

Para manejar un sistema completo de login y password con contraseñas cifradas, necesitamos un método que cifre esos strings que el usuario introduce como contraseña; tanto en el formulario de registro como en el del login, ya que al codificar una contraseña, después tenemos que decodificarla para comprobar que ambas contraseñas (la que introduce el usuario en el login y la que tenemos en

la base de datos) coincidan.

Necesitamos pues:

- password_hash() para almacenar la contraseña en la base de datos a la hora de hacer el INSERT
 - PASSWORD_DEFAULT almacenamos la contraseña usando el método de encriptación bcrypt
 - PASSWORD_BCRYPT almacenamos la contraseña usando el algoritmo CRYPT_BLOWFISH compatible con crypt()
- password_verify() para verificar el usuario y la contraseña

```
<?php
// Almacenando usuario y password en BD

$susu = $_POST["usuario"];
$pas = $_POST["password"];

$sql = "INSERT INTO usuarios(usuario, password) VALUES (:usuario, :
password)";

$sentencia = $conexion -> prepare($sql);

$isOk = $sentencia -> execute([
    "usuario" => $usu,
    "password" => password_hash($pas,PASSWORD_DEFAULT)
]);
```

Ahora que tenemos el usuario codificado y guardado en la base de datos, vamos a recuperarlo para poder loguearlo correctamente.

```
<?php
// Recuperando usuario y password en BD

$susu = $_POST["login"] ?? "";

$sql = "select * from usuarios where usuario = ?";

$sentencia = $conexion -> prepare($sql);
$sentencia -> execute([$usu]);

$usuario = $sentencia -> fetch();

if($usuario && password_verify($_POST['pass'], $usuario['password']))
{
    echo"OK!";
} else {
    echo"KO";
}
```

7 Ejercicios

7.1 Ejercicio 1

Realizar el MVC del ejercicio “Vamos a crear un CRUD de una única tabla task” del tema anterior.